

Cerințe Minime – RTOS (Real-Time Operating System)

Target & Toolchain

- **MCU profile:** ARM Cortex-M3/M4 (ex. STM32F4xx class)
 - **Rulează pe:** Renode sau QEMU (-M `netduinoplus2` / `stm32`), opțional pe microcontroller fizic
 - **Toolchain:** `arm-none-eabi-gcc`, `openocd/gdb` (dacă se folosește placă reală)
-

Cerințe funcționale minime

Boot & Tick

- Inițializare vector table, `Reset_Handler`, sistem de ceas.
 - Configurare **SysTick** (sau **TIMx**) pentru un tick periodic (implicit 1 kHz, configurabil).
-

Preemptive Fixed-Priority Scheduler

- **Ready lists** separate pentru fiecare prioritate.
 - Alegerea task-ului de rulat se face în timp **O(1)**.
 - **Context switch** declanșat la fiecare tick și la apelurile de tip **blocking**.
 - Cel puțin **4 niveluri de prioritate** și **3 task-uri concurente** demonstrabile.
-

Task API

- Funcții minime:

- `rtos_task_create(fn, prio, stack)`
 - `rtos_yield()`
 - `rtos_delay(ms)` (bazat pe tick counter)
 - `rtos_start()` pentru pornirea scheduler-ului
-

Primitive de sincronizare

- **Semafor binar și mutex** (cu *priority inheritance* sau *priority ceiling*).
 - **Blocking wait** cu timeout (măsurat în ticks).
 - **Producer/consumer** cu o **message queue** (cel puțin 8 elemente).
-

Time Services

- **Millisecond tick counter** și funcția `rtos_now()`.
 - **Soft timers:** `rtos_timer_start(period_ms, callback)` implementat în coada de final a întreruperii de tick (*tick ISR tail*).
-

Determinism

- Măsurarea și raportarea timpilor:
 - **Worst-case interrupt latency**
 - **Context switch time**
 - Demonstrație **Rate-Monotonic Scheduling (RMS)** cu 2 task-uri periodice (ex. 5 ms și 20 ms):
 - Se arată respectarea și încălcarea controlată a termenelor (*deadlines*) în funcție de gradul de încărcare (utilization).
-

Device I/O (Minimal HAL)

- **UART** (polled sau IRQ) pentru loguri.
 - **GPIO blink task** la 10 Hz pentru a demonstra că **scheduling-ul** rămâne funcțional sub sarcină.
 - Pe emulator: GPIO via **Renode**; pe microcontroller: **LED + button**.
-

Cerințe non-funcționale minime

- **Hard separation:** codul ISR trebuie să fie minimal și cu execuție limitată în timp; logica principală se realizează în context de task.
 - **Static allocation path** disponibil (sistemul RTOS trebuie să poată rula fără heap dinamic).
 - **Fișier de configurare:** `rtos_config.h` cu parametrii principali:
 - tick rate
 - număr maxim de task-uri
 - număr maxim de priorități
 - dimensiune stack per task
 - **Timing budget documentat:** specificați **WCET** (Worst-Case Execution Time) presupus pentru fiecare primitivă RTOS.
-

Testare

- **Headless test:** sistemul pornește kernel-ul, creează 3 task-uri (high / medium / low), rulează 5 secunde și capturează logul UART.
 - Demonstrație de **priority preemption**: task-ul high previne execuția celor medium/low până când se blochează.
 - **Priority inversion scenario**: task medium deține mutex-ul, high așteaptă, low rulează → apoi se arată că **priority inheritance / ceiling** rezolvă problema.

- **Queue throughput test:** producător/consumator – se verifică că toate mesajele sunt procesate fără pierderi la ratele configurate.
-

Livrabile

- **Repo Git**
 - **Demo app:**
 - T1 periodic la 5 ms (~0.8 ms execuție)
 - T2 periodic la 20 ms (~2 ms execuție)
 - T3 background logger
 - Log UART care marchează **dispatch-ul task-urilor** și **deadline-urile**.
 - **Documentație tehnică:**
 - **Timing table** cu perioade, execuții și priorități
 - **Latency data** (timpi măsurați pentru ISR și context switch)
 - **Plot** cu timpii de răspuns în funcție de încărcare
-

Minimal Demo Script

1. Boot → afișează tick rate; creează 3 task-uri.
 2. Task-urile T1 și T2 rulează periodic, se afișează timestamp-uri și jitter (în μ s).
 3. Se simulează **priority inversion** (med deține mutex, high așteaptă, low rulează) → se observă deadline miss;
apoi se activează **PI/ceiling**, iar deadline miss dispare.
 4. Se crește sarcina (load) până la apariția controlată a unor **deadline misses**.
-

Restricții

- NU importați cod din **FreeRTOS**, **Zephyr**, **RTX** – toate primitivele trebuie implementate manual.
 - NU se permit modificări dinamice de prioritate – doar **fixed-priority scheduling**.
 - Fără **file system** sau **networking** (nu sunt cerute pentru nota maximă).
-

Opțional (BONUS)

- **EDF Scheduler** (Earliest Deadline First) cu mecanism de **admission control**.
- **Event flags**, **memory pools**, **zero-copy queues**.
- **Tickless idle** / moduri de **low-power sleep**.
- Utilizarea **MPU** pentru protecția stack-urilor (*stack guard*).